FASE 1: Diagnóstico del Equipo y Entorno

1.1 Personal

Equipo de Desarrollo y Brechas de Conocimiento

Nombre	Rol	Fortalezas Técnicas	Debilidades / Brechas
Lucas	Project Manager	Conoce el flujo de trabajo Full Stack y tiene experiencia en proyectos.	Desconocimiento del entorno NodeJS. Me desvelo
Carlo	Front-end Engineer	Dominio de HTML, CSS y conocimientos básicos de JavaScript.	Falta de experiencia o conocimiento en React.
Mathew	Back-end Engineer	Conoce las bases de JavaScript.	Falta de experiencia o conocimiento en desarrollo Back-end.
Mario	Database Administrator	Conocimiento teórico en bases de datos.	Falta de experiencia en implementación práctica en un proyecto real.

Plan de Mitigación Semanal

- **Carlo:** Comenzar de inmediato con tutoriales de React para familiarizarse con la sintaxis de componentes y Hooks antes de tocar el código del proyecto.
- **Mathew:** Estudiar la arquitectura Back-end enfocándose en cómo se conectan las rutas (routers) con los controladores en Express/Node.js.
- **Mario:** Revisar casos de estudio y tutoriales sobre la implementación y migración de bases de datos relacionales en entornos de proyectos reales.
- **Lucas:** Usar su conocimiento previo de arquitecturas .NET hacia el ecosistema de Node.js.

1.2 Producto

Declaración de Alcance

El sistema resuelve el problema de la descentralización y falta de seguimiento en los reportes de fallas dentro del campus. Actualmente, las incidencias se pierden en comunicaciones informales, lo que genera tiempos de respuesta lentos y una nula capacidad para medir la eficiencia del personal. Este software centraliza todo el ciclo de vida de un desperfecto, desde su reporte con evidencia fotográfica hasta su resolución auditada.

La solución permite a los jefes de mantenimiento puedes asignar tareas basadas en la carga de trabajo real y especialidad de los técnicos. Al implementar un flujo de estados estricto y un sistema de escalado automático (SLA), el sistema garantiza que ningún reporte quede en el olvido, transformando el mantenimiento reactivo en una operación gestionada por datos y métricas de desempeño.

Finalmente, el producto ofrece una interfaz móvil para técnicos y usuarios, facilitando la captura de evidencia en el sitio y la confirmación de satisfacción. Con un registro de auditoría inmutable y notificaciones multicanal, el sistema asegura transparencia total en el uso de los recursos de la universidad.

Límites del Sistema

- **Gestión de Inventario Física:** El sistema permite comentar si faltan piezas (estado Bloqueado), pero no llevará el control de existencias, compras de almacén, ni proveedores.
- **Mantenimiento Preventivo Automatizado:** El sistema es puramente correctivo (basado en tickets de fallas actuales); no generará órdenes de trabajo automáticas basadas en calendarios o sensores IoT.
- **Gestión de Nómina o Pagos:** Aunque mide la eficiencia de los técnicos, no calculará salarios, horas extra, ni realizará pagos electrónicos al personal.

1.3 Proceso

Marco de Trabajo: Scrum/XP Hybrid

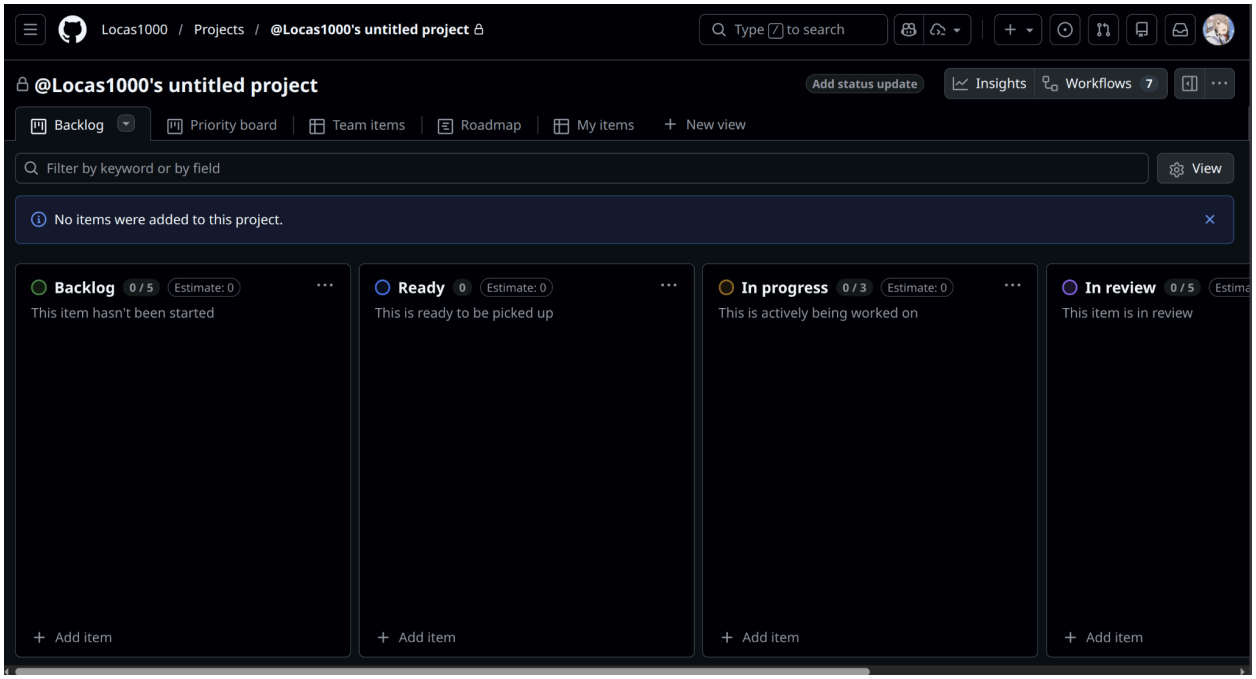
Estamos utilizando un marco Ágil basado en Scrum, complementado con prácticas de Extreme Programming (XP).

- **Scrum:** Dividimos el semestre en Sprints de 2 semanas para entregar incrementos funcionales constantes.
- **XP:** Implementamos Programación en Parejas ocasional y Revisiones de Código obligatorias para mitigar la curva de aprendizaje tecnológica del equipo.

Flujo de Trabajo Técnico

Utilizamos GitHub Flow para asegurar una rama principal estable:

1. **Refinamiento del Backlog:** Lucas (PM) desglosa requerimientos en GitHub Projects con criterios de aceptación.
2. **Creación de Rama:** Se crea una rama local partiendo de main con el commit feat/nombre-de-la-tarea.
3. **Desarrollo y Pruebas Locales:** El código se escribe y se verifica la compilación local sin romper funciones previas.
4. **Pull Request & Code Review:** Se sube la rama y se abre un PR. antes de avanzar.
5. **Pruebas de Integración (QA):** Mario o el encargado de QA validan el movimiento en la base de datos y respuesta de la UI.
6. **Merge a Main:** Se integra el código, se cierra el ticket y se notifica al equipo.



1.4 Proyecto (Project)

Herramienta de Gestión

Utilizamos **GitHub Projects**. Su integración nativa con nuestro repositorio permite vincular directamente los Issues con los Pull Requests.

Plan de Continuidad

Si un integrante se ausenta.

- **Wiki del Repositorio:** Documentación de arquitectura, diagramas de BD y lógica de módulos.
- **Issues/Tickets Detallados:** Criterios de aceptación explícitos y notas técnicas en cada

tarjeta para retomar el hilo del desarrollo sin fricciones.

FASE 2: Descomposición y Estimación

2.1 Estructura de Desglose de Trabajo

- **Nivel 1: Sistema de Gestión de Mantenimiento e Incidencias (SGM)**
  - **Nivel 2: Infraestructura y Seguridad (Responsable: Lucas)**
    - Nivel 3: Configuración del entorno de desarrollo (Docker/Linux).
    - Nivel 3: Implementación de Autenticación JWT.
    - Nivel 3: Sistema de Logs.
  - **Nivel 2: Persistencia y Base de Datos (Responsable: Mario)**
    - Nivel 3: Diseño del esquema relacional en PostgreSQL.
    - Nivel 3: Desarrollo de triggers para el histórico de cambios de estado.
    - Nivel 3: Script de migración y datos de prueba.
  - **Nivel 2: API Backend (Node.js)**
    - Nivel 3: Servidor base con Express.js.
    - Nivel 3: Integración de ORM (Prisma/Sequelize) para PostgreSQL.
    - Nivel 3: Middleware de autenticación
  - **Nivel 2: Interfaz de Usuario y Experiencia (Responsable: Carlo)**
    - Nivel 3: Formulario de levantamiento de tickets con acceso a cámara.
    - Nivel 3: Dashboard de Despacho para Jefes con métricas.
    - Nivel 3: Panel del Técnico para gestión de tarea

2.2 Estimación Pragmática

Tarea Crítica (Nivel 3)	Story Points	Justificación técnica
Motor de Workflow (States)	8	Manejar estados asíncronos en Node requiere mucho cuidado para evitar condiciones de carrera.
Auth JWT (Node.js)	5	Configurar middlewares de seguridad y proteger rutas

		de la API de forma centralizada.
Dashboard en React	5	El reto es el manejo de estados globales (Context API) para mostrar métricas en tiempo real.
CRUD de Tickets	3	Tarea estándar de Express, pero incluye la lógica de subida de imágenes (Multer).
Configuración de DB	3	Definir modelos en el ORM que reflejen las necesidades del proyecto.

FASE 3: Gestión de Riesgos (RMMM)

3.1 Identificación y Análisis

Riesgo	Descripción	Probabilidad	Impacto
Cuello de botella	solo hay un programador con experiencia Full Stack; si se bloquea, Mathew y Carlo se detienen.	Alta	Crítico
Inconsistencia de Sincronía	El horario de trabajo de Lucas puede dejar al equipo bloqueado durante el día esperando revisiones.	Alta	Moderado

Curva de aprendizaje (Node/React)	Carlo o Mathew podrían no implementar la lógica de Hooks/Middlewares a tiempo.	Media	Crítico
Falla en despliegue DB	Problemas de conectividad entre la API de Node.js y PostgreSQL en Docker o producción.	Baja	Crítico
Saturación por exámenes	La carga académica de otras materias reduce drásticamente las horas dedicadas al proyecto.	Alta	critico

3.2 Plan RMMM

Riesgo 1: Cuello de botella y Sincronía

- **Mitigación:** Implementar un "Contrato de Interfaz". Definir los endpoints de la API y el modelo de datos antes de programar para que Front y Back trabajen asíncronamente. Uso estricto de GitHub Issues detallados.
- **Monitoreo:** (Métrica) Si un Issue permanece en "In Progress" más de 3 días sin commits o comentarios nuevos.

Riesgo 2: Curva de aprendizaje técnica (Node.js/React)

- **Mitigación:** Pair Programming preventivo para corregir errores de JavaScript de forma temprana.
- **Monitoreo:** Pull Requests rechazados más de 3 veces por errores conceptuales básicos.

FASE 4: Monitoreo y Prevención del "Síndrome del 90%"

4.1 Definición de Hitos Binarios

1. **Hito 1: Arquitectura Base y Autenticación (Semana 4)**

    - **Métrica de Terminado:** La API responde correctamente en entorno local, procesa un login exitoso, devuelve un JWT válido y rechaza el acceso a rutas protegidas sin el token. Pruebas exitosas documentadas en Postman.
2. **Hito 2: Core Funcional - CRUD de Tickets (Semana 6)**
    - **Métrica de Terminado:** Es posible crear, leer, actualizar y eliminar un reporte de falla desde la API, incluyendo el almacenamiento y recuperación de un archivo de imagen, reflejándose correctamente en PostgreSQL.
3. **Hito 3: Integración Frontend y Flujo de Estados (Semana 9)**
    - **Métrica de Terminado:** La interfaz de React se comunica al 100% con la API. Un ticket puede transicionar por todos sus estados (Abierto -> Resuelto) haciendo clic en los botones de la interfaz sin arrojar errores de consola.
4. **Hito 4: Despliegue y Congelamiento de Código (Semana 12)**
    - **Métrica de Terminado:** El proyecto se levanta completamente en cualquier máquina utilizando docker-compose up sin configuraciones adicionales y la documentación en Swagger es accesible vía navegador.

4.2 Simulación de Control

Escenario: Semana 8 del semestre y solo se ha completado el 30% de las tareas planificadas.

Decisión Administrativa: Se ejecuta un **Recorte de Alcance**.

Justificación: Aumentar las horas de trabajo es irrealizable por la saturación de exámenes (riesgo ya mapeado) y bajar la calidad resultaría en un sistema inestable que reprueba la materia. Se eliminarán módulos accesorios (como el dashboard de gráficas complejas) para enfocar el 100% del esfuerzo restante en garantizar la calidad técnica del flujo principal de creación y cierre de tickets. Un sistema pequeño que funciona perfectamente se evalúa mejor que un sistema masivo lleno de errores.

FASE 5: Estrategia de Cierre y Post-Mortem

5.1 Criterios de Aceptación Final (Checklist de Cierre)

- **Validación del Flujo de Estados:** Un ticket recorre su ciclo completo (Abierto -> Asignado -> En Progreso -> Resuelto -> Confirmado -> Cerrado) sin errores de integridad relacional.
- **Manual de Despliegue:** Archivo README.md y docker-compose.yml listos para levantar BD, Backend y Frontend con un solo comando.
- **Documentación de la API:** Endpoints documentados con Swagger/OpenAPI.
- **Cobertura de Pruebas:** Rutas críticas (creación de tickets y estados) pasan pruebas de integración.
- **Limpieza de Repositorio:** Rama main limpia, sin archivos .env expuestos ni carpetas basura (node_modules), usando Conventional Commits.

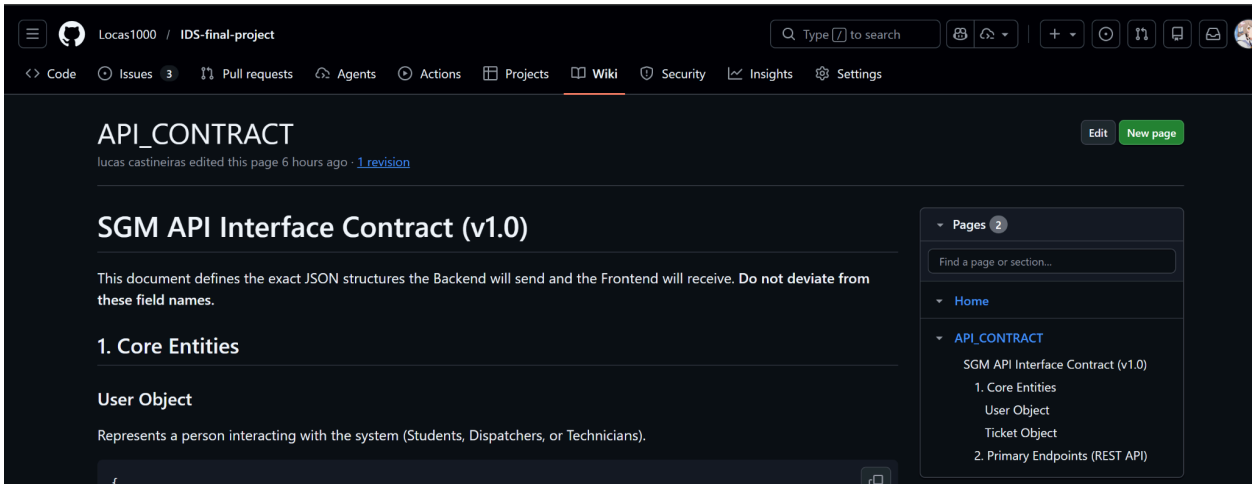
5.2 El Fantasma del Mantenimiento

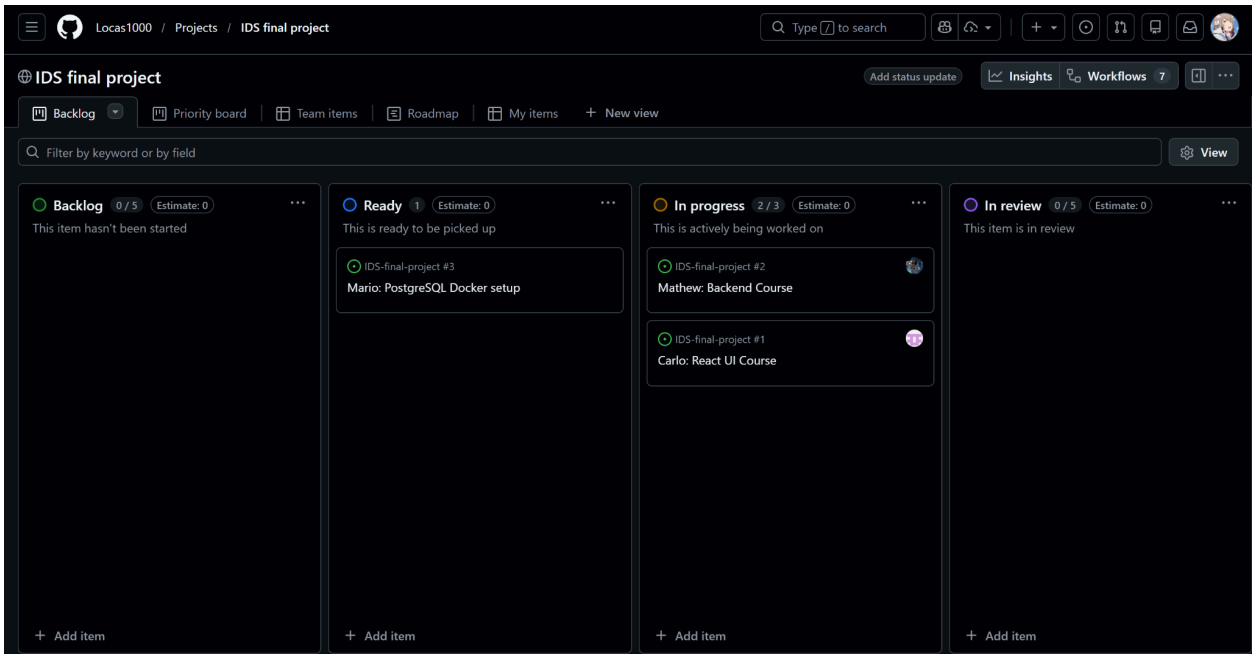
El elemento específico diseñado para el futuro mantenedor es nuestra **Arquitectura Documentada y Autodescriptiva (Swagger + Docker)**:

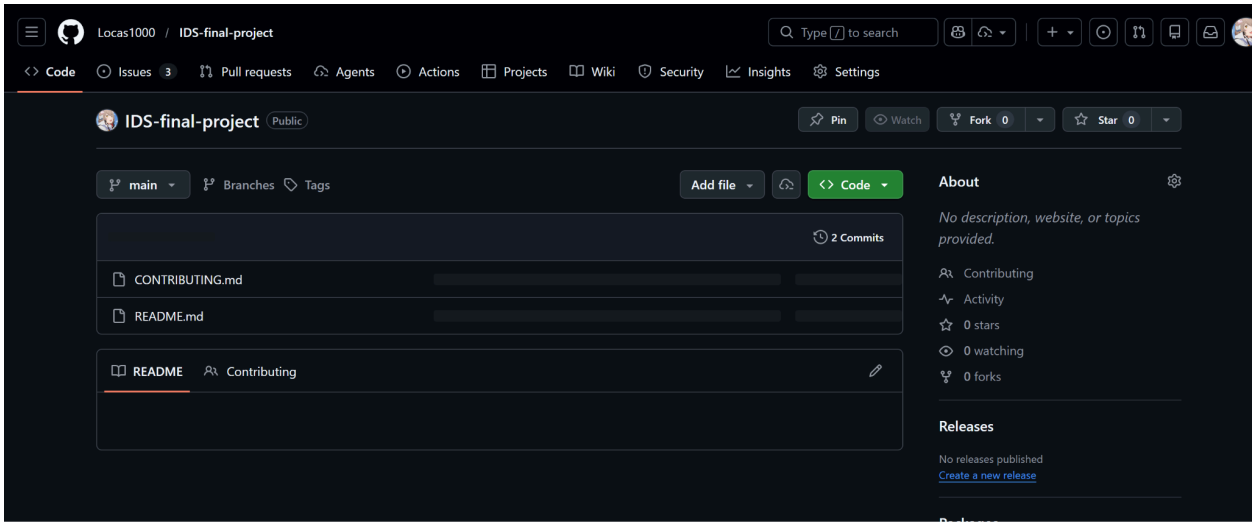
- **Dockerización:** El futuro ingeniero no lidiará con dependencias locales de Node o PostgreSQL; solo necesita Docker instalado.
- **Swagger UI:** Al acceder a /api-docs, el sucesor tendrá una interfaz visual interactiva para entender y probar qué parámetros exactos espera el backend, sin necesidad de ingeniería inversa.
- **Código Autodescriptivo:** El uso de JSDoc en el código de Node garantiza que el IDE del próximo desarrollador le explique la lógica de los SLAs mediante autocompletado, evitando depender de PDFs externos que se vuelven obsoletos rápidamente.

Repositorio

https://github.com/Locas1000/IDS-final-project




Locas1000 / Projects / IDS final project
Type / to search
IDS final project
Add status update
Insights
Workflows 7
Backlog
Priority board
Team items
Roadmap
My items
New view
Filter by keyword or by field
View
Backlog 0 / 5 Estimate: 0
This item hasn't been started
Add item
Ready 1 Estimate: 0
This is ready to be picked up
IDS-final-project #3
Mario: PostgreSQL Docker setup
Add item
In progress 2 / 3 Estimate: 0
This is actively being worked on
IDS-final-project #2
Mathew: Backend Course
IDS-final-project #1
Carlo: React UI Course
Add item
In review 0 / 5 Estimate: 0
This item is in review
Add item


Locas1000 / IDS-final-project
Type / to search
Code
Issues 3
Pull requests
Agents
Actions
Projects
Wiki
Security
Insights
Settings
IDS-final-project Public
Pin
Watch
Fork 0
Star 0
main
Branches
Tags
Add file
Code
2 Commits
CONTRIBUTING.md
README.md
README
Contributing
About
No description, website, or topics provided.
Contributing
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release